

Linear regression

$$y = \underline{W}^T \underline{x} + b$$

y: prediction

 $\underline{W}$: weight

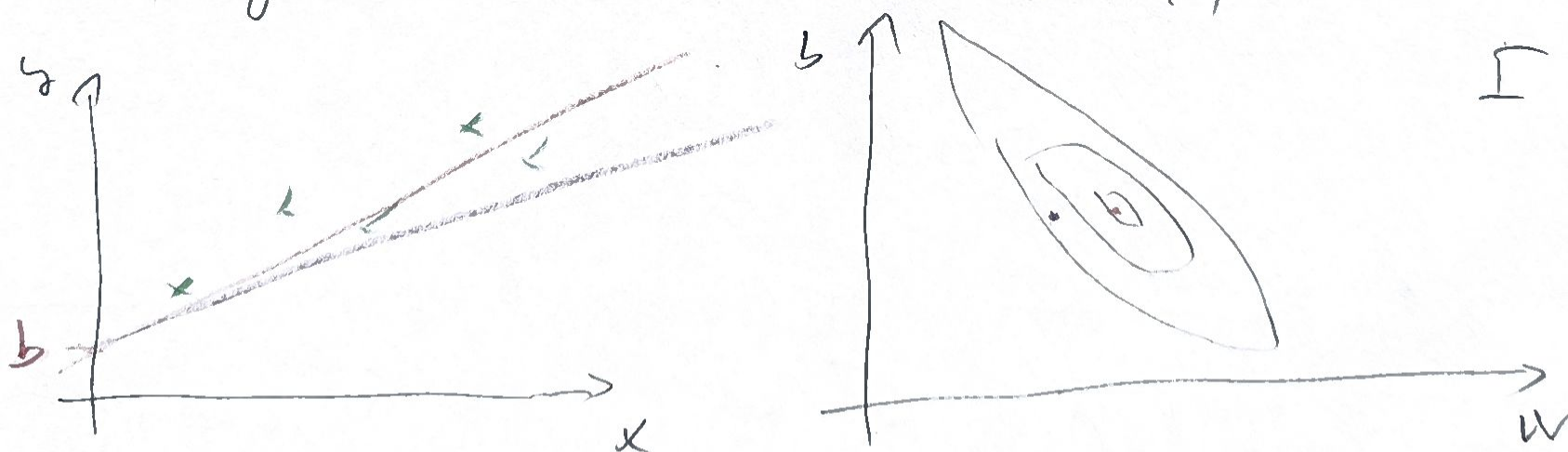
b: bias

Vectorization:

$$\underline{X} = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \ddots & \vdots \end{pmatrix} \begin{matrix} \leftarrow \text{one input vector} \\ \downarrow N \\ \rightarrow \text{one data point} \end{matrix}$$

↑
one feature

N: number of data points

set of $\{\underline{W}, b\}$ is called hypothesist: target for $\underline{x}$ Loss function: $L(y, t) = \frac{1}{2} \|y - t\|^2$ Cost function: $J(\underline{W}, b) = \frac{1}{2N} \sum_{i=1}^N \| \underline{W}^T \underline{x}^{(i)} + b - t^{(i)} \|^2$ Solution:

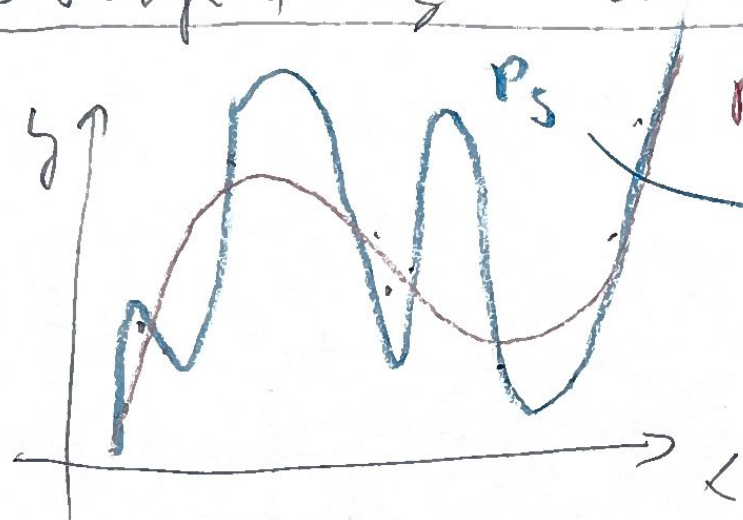
$$\underline{W} = \left(\underline{X}^T \underline{X} \right)^{-1} \underline{X}^T \cdot \underline{t}$$

Gradient descent

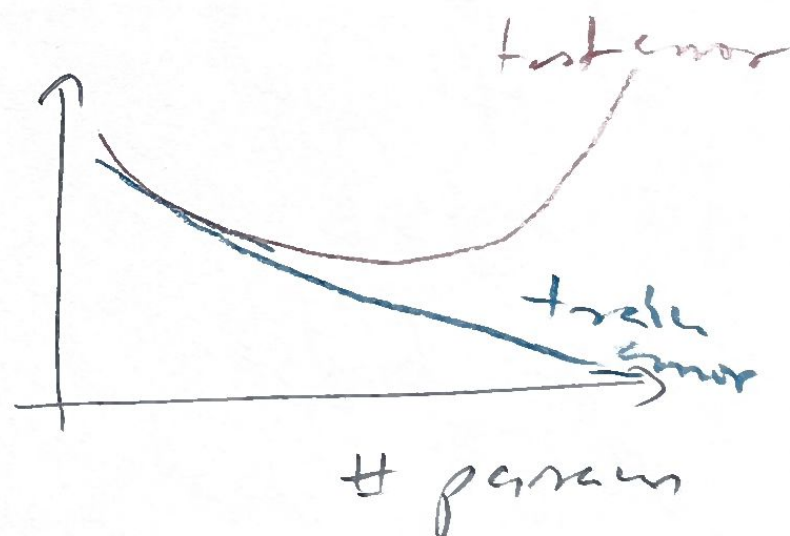
$$W_j^{(k+1)} = W_j^{(k)} - \alpha \frac{\partial I}{\partial W_j^{(k)}} = W_j^{(k)} - \frac{\alpha}{N} \sum_{i=1}^N (y_i^{(k)} - \hat{y}_i^{(k)}) x_{i,j}^{(k)}$$

α : learning rate

Overfitting and regularization



$P_3 \rightarrow \max(W) \approx 1$
 $\max(W) \approx 10$

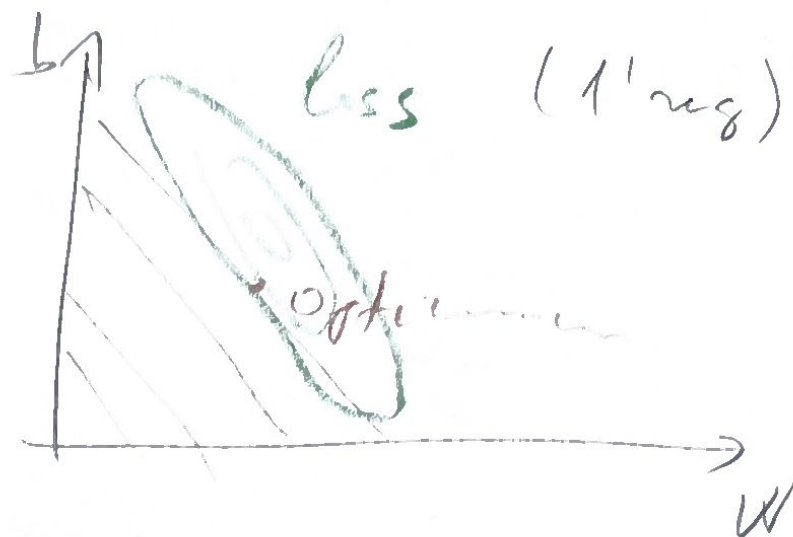
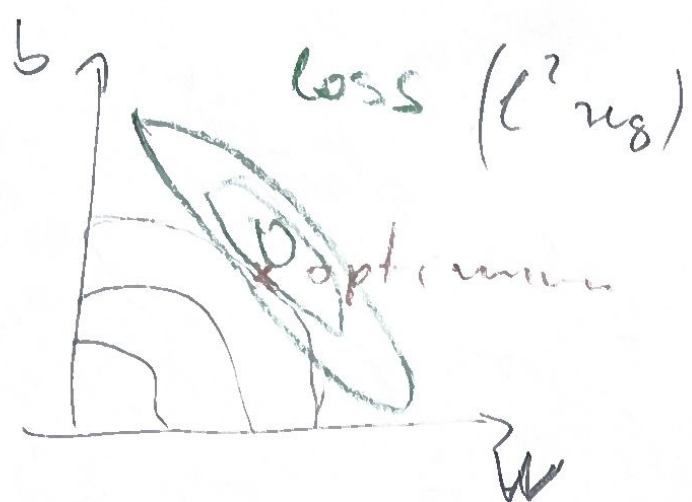


What if we used a lot of parameters

Regularization: force weights to be small

$$I_{reg} = I + \lambda_2 \cdot \sum_j W_j^2 \quad l^2 \text{ regularization}$$

$$I_{reg} = I + \lambda_1 \cdot \sum_j |W_j| \quad l^1 \text{ regularization}$$



Gradient descent

$$\begin{aligned}
 W_i^{(t+1)} &= W_i^{(t)} - \alpha \frac{\partial J_{reg}}{\partial W_i} = \\
 &= W_i^{(t)} - \alpha \frac{\partial J}{\partial W_i^{(t)}} - \alpha \lambda \frac{\partial \ell}{\partial W_i^{(t)}} = \\
 &= (1 - 2\lambda) W_i^{(t)} - \alpha \frac{\partial J}{\partial W_i^{(t)}}
 \end{aligned}$$

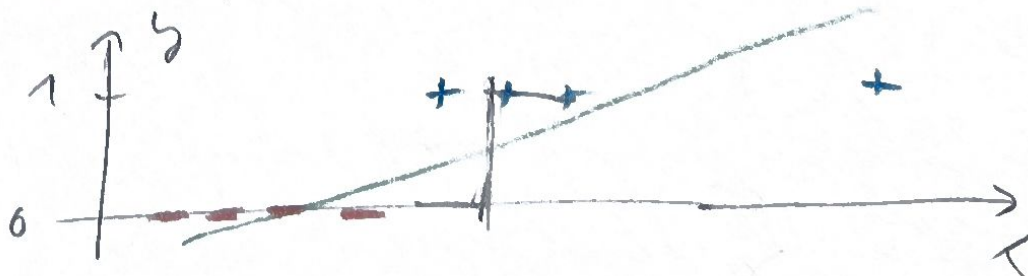
Binary data

$$\begin{aligned}
 z &= W^T x + b \\
 t &\in \{0, 1\}
 \end{aligned}$$

$$y = \begin{cases} 1 & \text{if } z \geq \nu \\ 0 & \text{otherwise} \end{cases}$$

Problem with the loss:

$$-L = \frac{1}{2} (y - t)^2$$

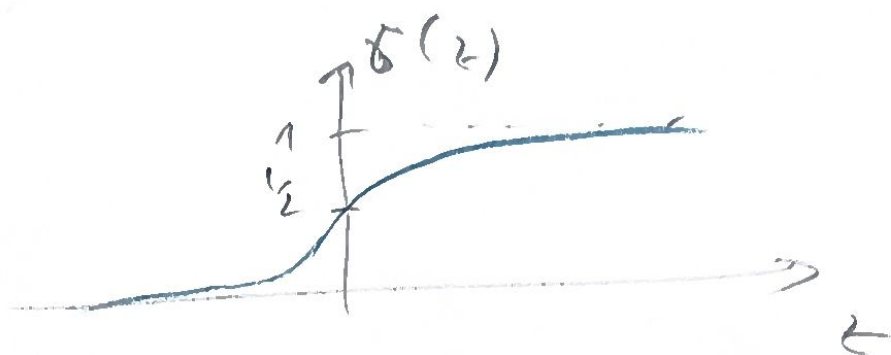


$$-L_b = \begin{cases} 0 & \text{if } y = t \\ 1 & \text{if } y \neq t \end{cases}$$

→ there is no gradient

Solution logistic activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

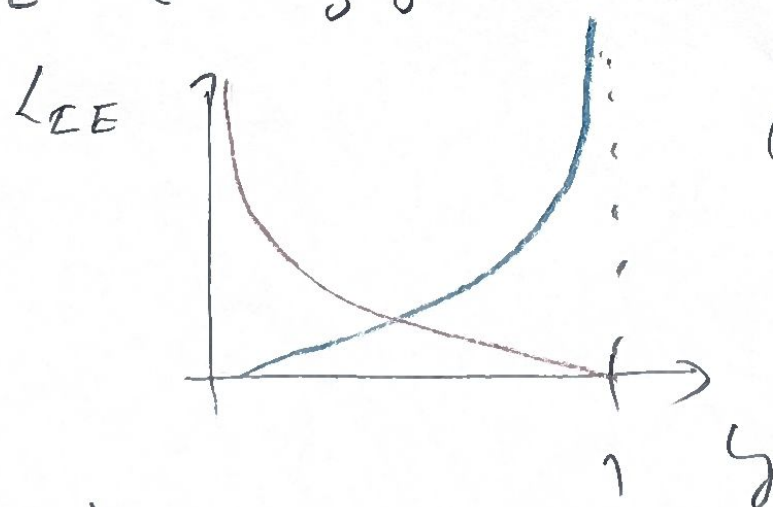


The loss function is still problematic,
the gradient vanishes for $z \gg 0$, $z \ll 0$

Cross entropy loss:

$$L_{CE} = \begin{cases} -\log y & \text{if } t=1 \\ -\log(1-y) & \text{if } t=0 \end{cases}$$

$$L_{CE} = -t \cdot \log y - (1-t) \log(1-y)$$



largest gradient
if farthest.

Logistic regression

$$z = W^T x + b$$

$$y = \sigma(z)$$

$$L_{CE} = -t \log y - (1-t) \log(1-y)$$

$$W^{(t+1)} = W^{(t)} - \frac{2}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x^{(i)} \quad (\text{as for linear})$$

Multiclass classification

- Target is a discrete set $t^{(i)} \in \{1, 2, \dots, k\}$
- one hot encoding representation:

$$\underline{t}^{(i)} = (0, 0, \dots, 0, 1, 0, \dots, 0)$$

k values only $k = t^{(i)} + 1$

Vectorised linear predictions:

$$\underline{z}^{(i)} = \underline{W} \underline{x}^{(i)} + \underline{b}$$

$$z_k = \sum_j W_{kj} x_j^{(i)} + b_k$$

Softmax function

$$y_k = \text{softmax}(z_1, \dots, z_k)_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}$$

↑
logits

Properties:

$$\sum_k y_k = 1, \quad y_k > 0$$

Cross categorical entropy loss:

$$\begin{aligned} L_{CE}(y, t) &= - \sum_{k=1}^k t_k \log y_k = \\ &= - \underline{t}^T (\log y) \end{aligned}$$

elementwise
log

Softmax regression

$$\underline{z} = \underline{W} \underline{x} + \underline{b}$$

$$\underline{y} = \text{softmax}(\underline{z})$$

$$L_{\text{ce}} = - \underline{t}^T (\log \underline{y})$$

$$\frac{\partial L_{\text{ce}}}{\partial \underline{z}}$$

$$= \underline{y} - \underline{t} \rightarrow \text{used to calculate gradients}$$